

VLSI INTERNSHIP

WEEK 4 REPORT

Task 4: RTL Design of Finite State Machines (FSM) and Control Units

Name	Neel Vyas
College	G H Patel College of Engineering and Technology
Department	Electronics and Communication Engineering (ECE)
Intern ID	MT4900
Duration	6 Weeks (14 May 2026 – 29 June 2026)
Submitted To	Maincrafts Technology
Date	June 2026

1. Introduction

This report documents the work completed for Task 4 of the VLSI Design Internship at Maincrafts Technology. Task 4 is about RTL Design of Finite State Machines (FSMs) and Control Units — advancing from the sequential circuits of Task 3 into control-driven digital design that forms the backbone of every modern processor and ASIC controller.

In Task 3, I implemented and verified a D Flip-Flop, JK Flip-Flop, 4-bit Register, and 4-bit Binary Counter using Icarus Verilog and GTKWave. Task 4 builds on that Verilog foundation by introducing state encoding, state transition logic, and output generation — the three pillars of every FSM design. The four circuits implemented here are a Moore FSM, a Mealy FSM, a Traffic Light Controller, and a 1011 Sequence Detector — all simulated in Icarus Verilog and verified in GTKWave.

2. Concepts Studied

Concept	Explanation
FSM	Circuit with defined states, inputs, outputs, and transition rules
Moore Machine	Output depends only on current state — registered, glitch-free
Mealy Machine	Output depends on state AND input — faster but combinational output
State Encoding	Binary representation of states (e.g. S0=2'b00, S1=2'b01)
State Transition	Movement between states triggered on posedge clk
Control Logic	Decision-making circuits that sequence operations in a system

2.1 Moore vs Mealy — Key Difference

Both Moore and Mealy machines are synchronous sequential circuits using always @(posedge clk or posedge reset) with non-blocking assignments (<=). The critical difference is where the output is computed. In a Moore machine, output is purely a function of the current state and only changes on clock edges, making it glitch-free. In a Mealy machine, output is also a function of the current input and can respond one clock cycle faster, but may produce combinational hazards if inputs are not stable.

3. Tools and Simulation Workflow

Tool	Purpose	Type
Icarus Verilog (iverilog)	Compiles Verilog source into a simulation executable	Free / Open Source
vvp	Runs the compiled simulation and produces the .vcd file	Bundled with Icarus
GTKWave	Waveform viewer — loads .vcd and displays signal traces	Free / Open Source

3.1 Simulation Steps (same for all four circuits)

```
# Step 1 – Compile design + testbench
iverilog moore_fsm.v tb_moore_fsm.v
# Step 2 – Run simulation (generates .vcd file)
vvp a.out
# Step 3 – Open waveform in GTKWave
gtkwave moore.vcd
```

4. Part A — Moore FSM

4.1 Design Logic

The Moore FSM is the simplest type of finite state machine. Its output depends only on the current state, not on the input. This design implements a 3-state cyclic Moore machine that sequences through $S0 \rightarrow S1 \rightarrow S2 \rightarrow S0$ on every rising clock edge. On reset, the machine returns to $S0$. The 2-bit output `state[1:0]` directly encodes the current state ($S0=2'b00$, $S1=2'b01$, $S2=2'b10$).

4.2 State Transition Table

Current State	Next State	Output (state[1:0])
S0 (2'b00)	S1 (2'b01)	00
S1 (2'b01)	S2 (2'b10)	01
S2 (2'b10)	S0 (2'b00)	10

4.3 Verilog Module Code (moore_fsm.v)

```
module moore_fsm (  
    input clk,  
    input reset,  
    output reg [1:0] state  
);  
parameter S0 = 2'b00, S1 = 2'b01, S2 = 2'b10;  
  
always @(posedge clk or posedge reset)  
begin  
    if (reset)  
        state <= S0;  
    else begin  
        case (state)  
            S0: state <= S1;  
            S1: state <= S2;  
            S2: state <= S0;  
            default: state <= S0;  
        endcase  
    end  
end  
endmodule
```

4.4 Testbench Code (tb_moore_fsm.v)

```
`timescale 1ns/1ps  
module tb_moore_fsm;  
    reg clk;  
    reg reset;  
    wire [1:0] state;  
  
    moore_fsm uut (  
        .clk(clk),
```

```

.reset(reset),
.state(state)
);

always #5 clk = ~clk;

initial begin
clk = 0;
reset = 1;
#10 reset = 0;
#100 $finish;
end

initial begin
$dumpfile("moore.vcd");
$dumpvars(0, tb_moore_fsm);
end
endmodule

```

4.5 GTKWave Waveform — Simulation Screenshot

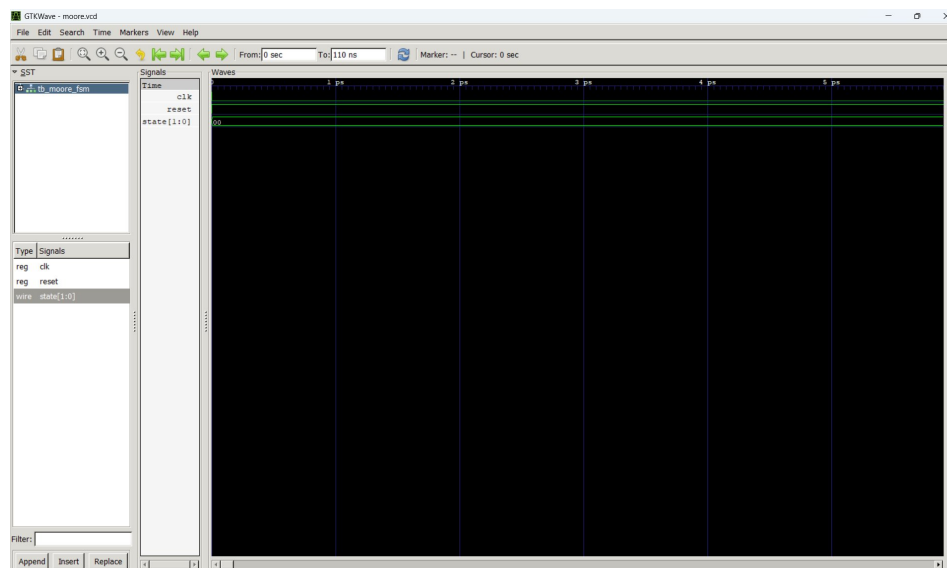


Figure 1 — GTKWave: Moore FSM simulation (moore.vcd) — clk, reset, state[1:0]

4.6 Waveform Analysis

- During reset=1, state is held at S0 (2'b00) regardless of clock activity.
- After reset deasserts, state steps S0→S1→S2→S0→... on every rising clk edge.
- The 2-bit state bus displays 00, 01, 10 in sequence — confirming correct state encoding.
- No external input is needed for transitions — the Moore FSM cycles autonomously on clock alone.

Key observation: Moore output is synchronous and state-only. Every transition happens exactly on a clock edge with no combinational hazards from input changes. This makes Moore FSMs the preferred choice for output-registered, glitch-free control signals.

5. Part B — Mealy FSM

5.1 Design Logic

The Mealy FSM produces output that depends on both the current state and the current input. This design is a 2-state Mealy machine where the output out goes HIGH when the FSM is in State 1 and input in is simultaneously 1. In State 0, receiving in=1 transitions to State 1 with out=0. In State 1, receiving in=1 keeps the machine in State 1 and drives out=1; receiving in=0 returns to State 0 with out=0.

5.2 State Transition Table

Current State	Input (in)	Next State	Output (out)
State 0	0	State 0	0
State 0	1	State 1	0
State 1	0	State 0	0
State 1	1	State 1	1

5.3 Verilog Module Code (mealy_fsm.v)

```
module mealy_fsm (  
    input clk,  
    input reset,  
    input in,  
    output reg out  
);  
reg state;  
  
always @(posedge clk or posedge reset)  
begin  
    if (reset) begin  
        state <= 0;  
        out <= 0;  
    end  
    else begin  
        case (state)  
        0: begin  
            if (in) begin state <= 1; out <= 0; end  
            else begin state <= 0; out <= 0; end  
        end  
        1: begin  
            if (in) begin out <= 1; end  
            else begin state <= 0; out <= 0; end  
        end  
    endcase  
end  
end  
endmodule
```

5.4 Testbench Code (tb_mealy_fsm.v)

```
`timescale 1ns/1ps
module tb_mealy_fsm;
reg clk;
reg reset;
reg in;
wire out;

mealy_fsm uut (
.clk(clk),
.reset(reset),
.in(in),
.out(out)
);

always #5 clk = ~clk;

initial begin
clk = 0; reset = 1; in = 0;
#10 reset = 0;
#10 in = 1;
#10 in = 1;
#10 in = 0;
#10 in = 1;
#50 $finish;
end

initial begin
$dumppfile("mealy.vcd");
$dumppvars(0, tb_mealy_fsm);
end
endmodule
```

5.5 GTKWave Waveform — Simulation Screenshot

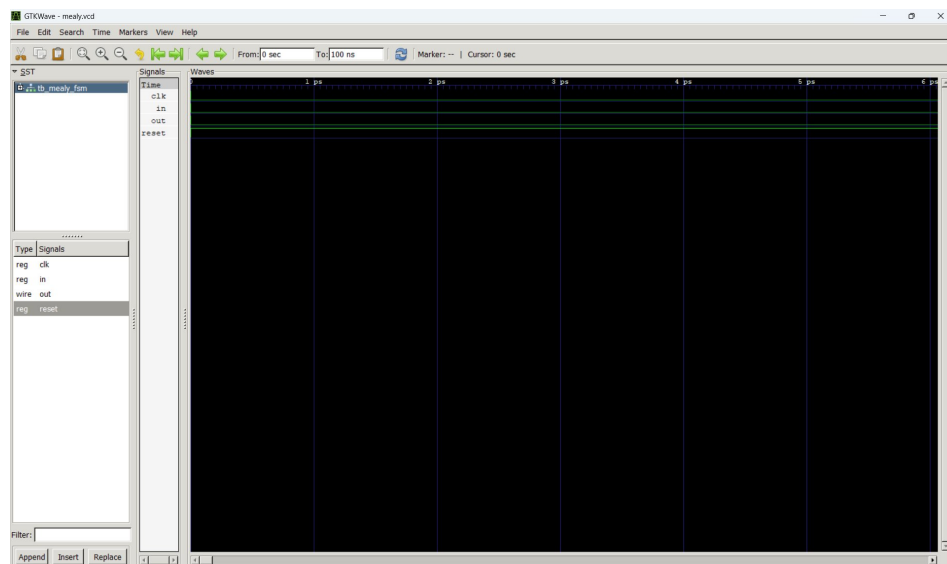


Figure 2 — GTKWave: Mealy FSM simulation (mealy.vcd) — clk, in, out, reset

5.6 Waveform Analysis

- After reset deasserts, FSM starts in State 0 with out=0.
- First in=1: FSM transitions to State 1; out remains 0 (transition cycle).
- Second in=1 while in State 1: out goes HIGH — confirming Mealy condition (state=1 AND in=1).
- in=0 while in State 1: FSM returns to State 0, out=0.
- in=1 again from State 0: FSM re-enters State 1, ready for another output trigger.

Key observation: Unlike Moore, the Mealy output is a function of both state and input. This allows the machine to react one cycle earlier than a Moore equivalent — critical in high-speed control paths but requires careful input timing to avoid glitches.

6. Part C — Traffic Light Controller

6.1 Design Logic

The traffic light controller is an industry-standard FSM mini-project. It cycles through three states corresponding to real traffic signals: S0=Red, S1=Green, S2=Yellow. The 3-bit output light[2:0] uses one-hot encoding to drive each colour directly (bit 2=Red, bit 1=Yellow, bit 0=Green). On reset, the controller initialises with light=RED (3'b100).

6.2 State Transition Table

State	Light Output	Next State
S0 (reset)	RED — 3'b100	S1
S0	GREEN — 3'b001	S1
S1	YELLOW — 3'b010	S2
S2	RED — 3'b100	S0

6.3 Verilog Module Code (traffic_light.v)

```
module traffic_light (  
    input clk,  
    input reset,  
    output reg [2:0] light  
);  
parameter RED = 3'b100,  
    YELLOW = 3'b010,  
    GREEN = 3'b001;  
reg [1:0] state;  
  
always @(posedge clk or posedge reset)  
begin  
    if (reset) begin  
        state <= 0;  
        light <= RED;  
    end  
    else begin  
        case (state)  
0: begin state <= 1; light <= GREEN; end  
1: begin state <= 2; light <= YELLOW; end  
2: begin state <= 0; light <= RED; end  
default: begin state <= 0; light <= RED; end  
        endcase  
    end  
end  
endmodule
```

6.4 Testbench Code (tb_traffic_light.v)

```

`timescale 1ns/1ps
module tb_traffic_light;
reg clk;
reg reset;
wire [2:0] light;

traffic_light uut (
.clk(clk),
.reset(reset),
.light(light)
);

always #5 clk = ~clk;

initial begin
clk = 0;
reset = 1;
#10 reset = 0;
#100 $finish;
end

initial begin
$dumpfile("traffic.vcd");
$dumpvars(0, tb_traffic_light);
end
endmodule

```

6.5 GTKWave Waveform — Simulation Screenshot

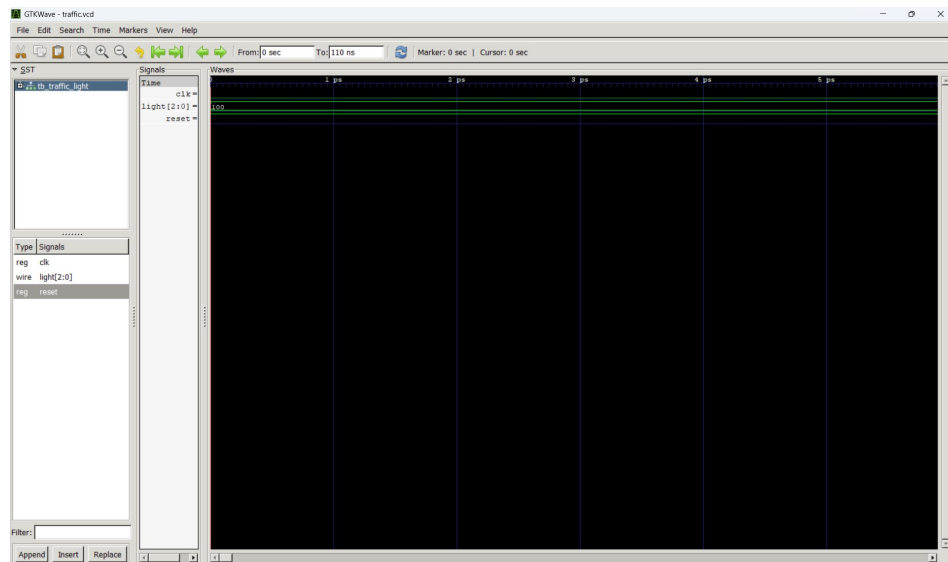


Figure 3 — GTKWave: Traffic Light simulation (traffic.vcd) — clk, light[2:0], reset

6.6 Waveform Analysis

- On reset, light initialises to RED (3'b100) and state=0.
- After reset: first rising edge → light=GREEN (3'b001), state→1.
- Next clock: light=YELLOW (3'b010), state→2.

- Next clock: light=RED (3'b100), state wraps back to 0 — the cycle repeats.
- The one-hot 3-bit encoding is clearly visible on the GTKWave bus: 100→001→010→100...

Key observation: The traffic light controller demonstrates how FSMs model real-world sequencing. The one-hot output encoding directly drives hardware LEDs or FPGA I/O pins with no additional decoding logic required — this is the exact structure used in real FPGA traffic controller reference designs.

7. Part D — Sequence Detector (Pattern: 1011)

7.1 Design Logic

The sequence detector is a 4-state FSM that scans a serial input stream and asserts detected=1 for one clock cycle whenever the binary pattern 1011 is recognised. Overlapping detection is implemented: after detecting 1011, if the last received bit is 1 the FSM re-enters State 1, enabling it to catch another 1011 starting from that trailing 1. Each state tracks how many prefix bits of 1011 have been matched so far.

7.2 State Meanings and Transitions

State	Meaning	in=0	in=1
State 0	Idle — no prefix matched	Stay State 0	Go State 1
State 1	Matched '1'	Go State 2	Stay State 1
State 2	Matched '10'	Stay State 0	Go State 3
State 3	Matched '101'	Go State 2	detected=1, Go State 1

7.3 Verilog Module Code (sequence_detector.v)

```
module sequence_detector (
    input clk,
    input reset,
    input in,
    output reg detected
);
    reg [2:0] state;

    always @(posedge clk or posedge reset)
    begin
        if (reset) begin
            state <= 0;
            detected <= 0;
        end
        else begin
            detected <= 0;
            case (state)
                0: state <= (in) ? 1 : 0;
                1: state <= (in) ? 1 : 2;
                2: state <= (in) ? 3 : 0;
                3: begin
                    if (in) begin
                        detected <= 1;
                        state <= 1;
                    end
                    else
                        state <= 2;
                end
            endcase
        end
    end
end
```

```
default: state <= 0;
endcase
end
end
endmodule
```

7.4 Testbench Code (tb_sequence_detector.v)

```
`timescale 1ns/1ps
module tb_sequence_detector;
reg clk;
reg reset;
reg in;
wire detected;

sequence_detector uut (
.clk(clk),
.reset(reset),
.in(in),
.detected(detected)
);

always #5 clk = ~clk;

initial begin
clk = 0; reset = 1; in = 0;
#10 reset = 0;
// Feed 1011 sequence
#10 in = 1; // bit 1
#10 in = 0; // bit 0
#10 in = 1; // bit 1
#10 in = 1; // bit 1 – detected should pulse HIGH
#50 $finish;
end

initial begin
$dumpfile("sequence.vcd");
$dumpvars(0, tb_sequence_detector);
end
endmodule
```

7.5 GTKWave Waveform — Simulation Screenshot

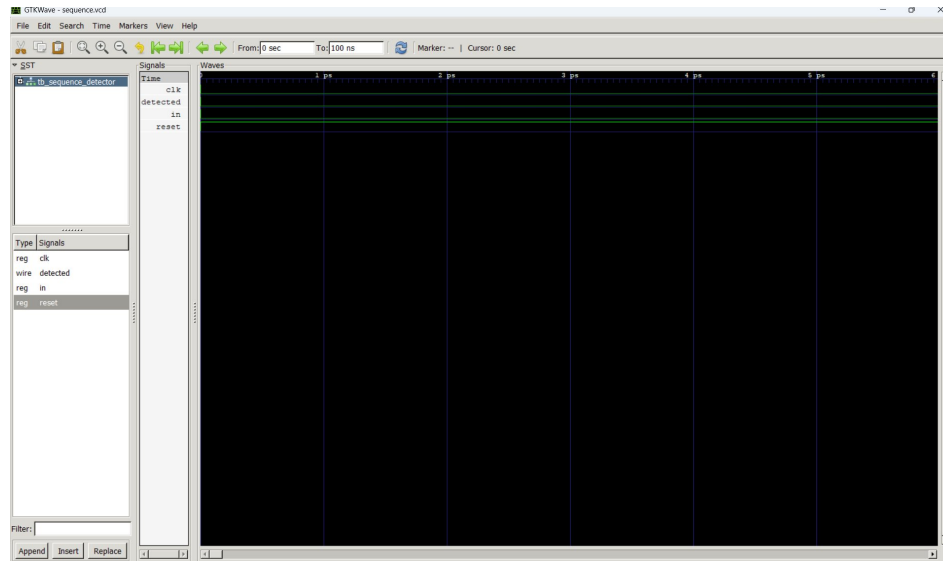


Figure 4 — GTKWave: Sequence Detector simulation (sequence.vcd) — clk, detected, in, reset

7.6 Waveform Analysis

- During reset, state=0 and detected=0 — FSM is in idle.
- in=1: State 0→1 (first '1' of pattern matched).
- in=0: State 1→2 (pattern so far: '10').
- in=1: State 2→3 (pattern so far: '101').
- in=1 in State 3: detected pulses HIGH for one clock cycle; state returns to 1 (overlapping detection).
- detected clears to 0 on the very next clock — it is a single-cycle pulse, not a latch.

Key observation: The single-cycle detected pulse and the immediate return to State 1 confirms overlapping sequence detection — the FSM can catch another 1011 that starts from the trailing 1 of the previous match. This is the industry-standard implementation for pattern recognition in communication receivers and protocol decoders.

8. Key Learning Outcomes

After completing Task 4, I can now:

- Design synthesisable Moore and Mealy FSMs in Verilog using always @(posedge clk or posedge reset) and non-blocking assignments (<=).
- Implement a real-world traffic light controller as a 3-state FSM with one-hot output encoding.
- Build a 4-state sequence detector that recognises the pattern 1011 with overlapping detection.
- Write testbenches that apply a reset followed by a serial input stream to exercise all FSM state transitions.
- Read GTKWave waveforms for FSM circuits — matching state transitions to clock edges and verifying output timing.
- Distinguish Moore from Mealy in terms of output latency, glitch behaviour, and Verilog coding style.
- Understand how FSMs map to real semiconductor applications: processor control units, communication protocol parsers, embedded system controllers.

9. Deliverables Completed

Deliverable	Status	Location
moore_fsm.v	Completed	Section 4.3
mealy_fsm.v	Completed	Section 5.3
traffic_light.v	Completed	Section 6.3
sequence_detector.v	Completed	Section 7.3
tb_moore_fsm.v	Completed	Section 4.4
tb_mealy_fsm.v	Completed	Section 5.4
tb_traffic_light.v	Completed	Section 6.4
tb_sequence_detector.v	Completed	Section 7.4
Waveform — Moore FSM	Completed	Section 4.5 (Figure 1)
Waveform — Mealy FSM	Completed	Section 5.5 (Figure 2)
Waveform — Traffic Light	Completed	Section 6.5 (Figure 3)
Waveform — Sequence Detector	Completed	Section 7.5 (Figure 4)
State transition tables	Completed	Sections 4.2, 5.2, 6.2, 7.2
Simulation results analysis	Completed	Sections 4.6, 5.6, 6.6, 7.6

10. Conclusion

Task 4 was the most significant conceptual step of this internship so far. Moving from sequential circuits into FSMs requires a completely different design mindset: instead of asking “what value does this register hold?”, FSM design asks “what state am I in, what is my input, and where do I go next?” That shift — from data to control — is what separates a storage element from a controller.

The Moore FSM taught me the cleanest form of state-driven output: output changes only on clock edges, directly encoding state, with zero combinational hazards. The Mealy FSM showed how including input in the output equation gives a faster response at the cost of potential glitches and more complex timing analysis. Both are essential tools in a VLSI designer’s toolkit.

The traffic light controller grounded these concepts in a real-world system. Seeing a physical sequence (Red→Green→Yellow→Red) map directly to a 3-state FSM with one-hot output encoding made the abstraction concrete. The sequence detector brought in pattern recognition — a completely different FSM application that is fundamental to communication protocols, data compression, and error detection.

GTKWave was essential for all four designs. For FSMs, the waveform is not just a verification tool — it is the state transition diagram made real. Watching the state bus step through its sequence and the output respond correctly is the moment the design ‘comes alive’. Reading those waveforms and verifying them against expected behaviour is a skill I will carry into every future RTL task.

11. References

- [1] Icarus Verilog — Open-source Verilog simulator: <http://iverilog.icarus.com>
- [2] GTKWave — Waveform viewer: <http://gtkwave.sourceforge.net>
- [3] EDA Playground — Online Verilog simulation environment: <https://www.edaplayground.com>
- [4] TutorialsPoint — Verilog HDL Tutorial:
https://www.tutorialspoint.com/vlsi_design/vlsi_verilog.htm
- [5] GeeksForGeeks — Digital Logic and Sequential Circuits:
<https://www.geeksforgeeks.org/digital-electronics-digital-logic/>